



UNIVERSIDADE FEDERAL DO CEARÁ
Campus Quixadá
Curso: Redes de Computadores
Disciplina: Redes

Aula 11 – Transporte Orientado a Conexão (TCP)

Prof. Rafael Braga

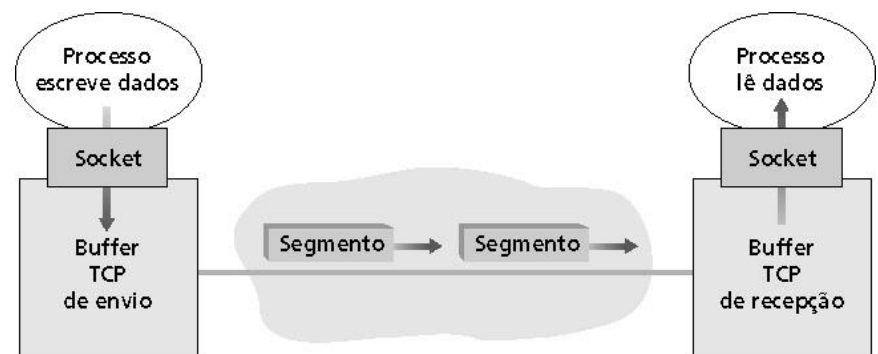
Conteúdo do Capítulo 3

- 3.1 Serviços da camada de transporte
- 3.2 Multiplexação e demultiplexação
- 3.3 Transporte não orientado para conexão: UDP
- 3.4 Princípios da transferência confiável de dados
- 3.5 Transporte orientado para conexão: TCP
 - estrutura do segmento
 - transferência confiável
 - controle de fluxo
 - gerenciamento de conexões
- 3.6 Princípios de controle de congestionamento
- 3.7 Controle de congestionamento do TCP

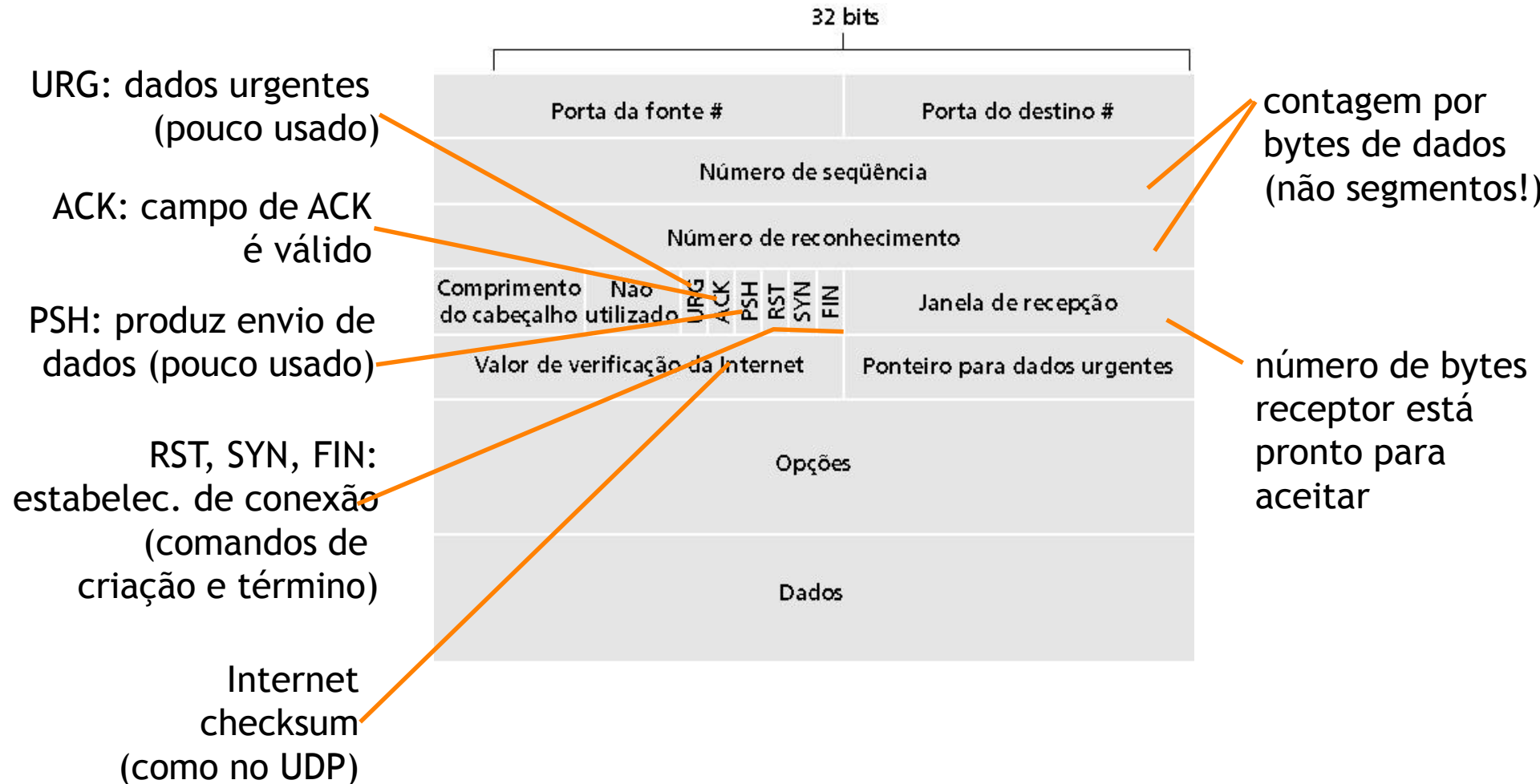
TCP: Visão geral

RFCs: 793, 1122, 1323, 2018, 2581

- **transmissão full duplex:**
 - fluxo de dados bi-direcional na mesma conexão
 - MSS: tamanho máximo de segmento
- **orientado a conexão:**
 - handshaking (troca de msgs de controle) inicia estado do transmissor e do receptor antes da troca de dados
- **fluxo controlado:**
 - receptor não será afogado pelo transmissor
- **ponto a ponto:**
 - 1 transmissor, 1 receptor
- **fluxo de bytes, ordenados, confiável:**
 - não estruturado em msgs
- **com paralelismo (*pipelined*):**
 - tam. da janela ajustado por controle de fluxo e congestionamento do TCP
- ***buffers de transmissão e recepção***



Estrutura do segmento TCP



TCP: nos. de seq. e ACKs

Nos. de seq.:

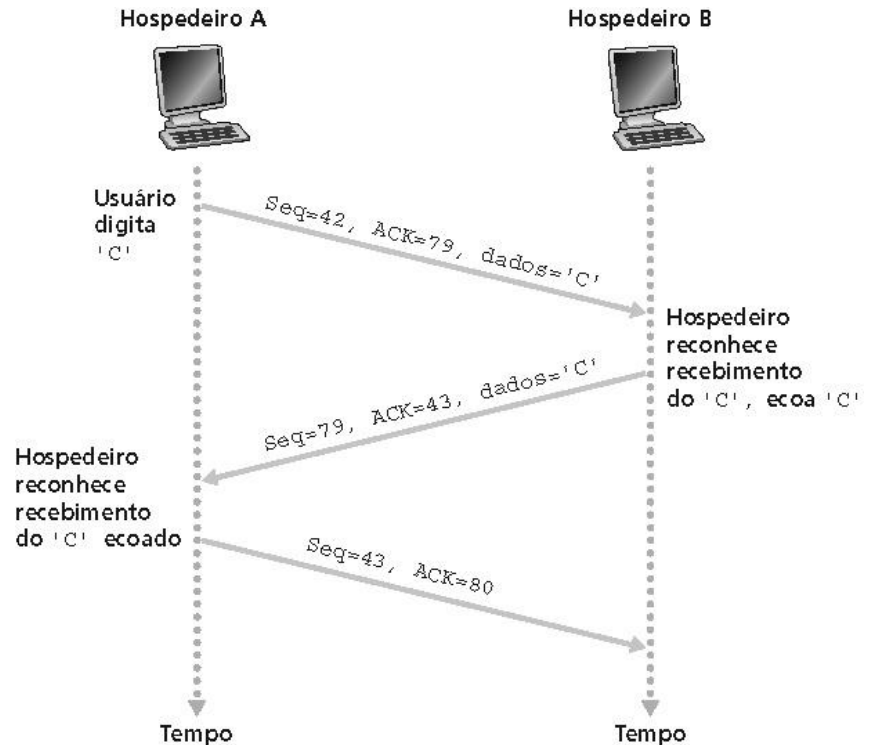
- “número” dentro do fluxo de bytes do primeiro byte de dados do segmento

ACKs:

- no. de seq do próx. byte esperado do outro lado
- ACK cumulativo

P: como receptor trata segmentos fora da ordem?

- R: espec do TCP omissa - deixado ao implementador



TCP: Tempo de Resposta (RTT – *Round Trip Time*) e Temporização

P: como escolher valor do temporizador TCP?

- maior que o RTT
 - note: RTT varia
- muito curto: temporização prematura
 - retransmissões são desnecessárias
- muito longo: reação demorada à perda de segmentos

P: como estimar RTT?

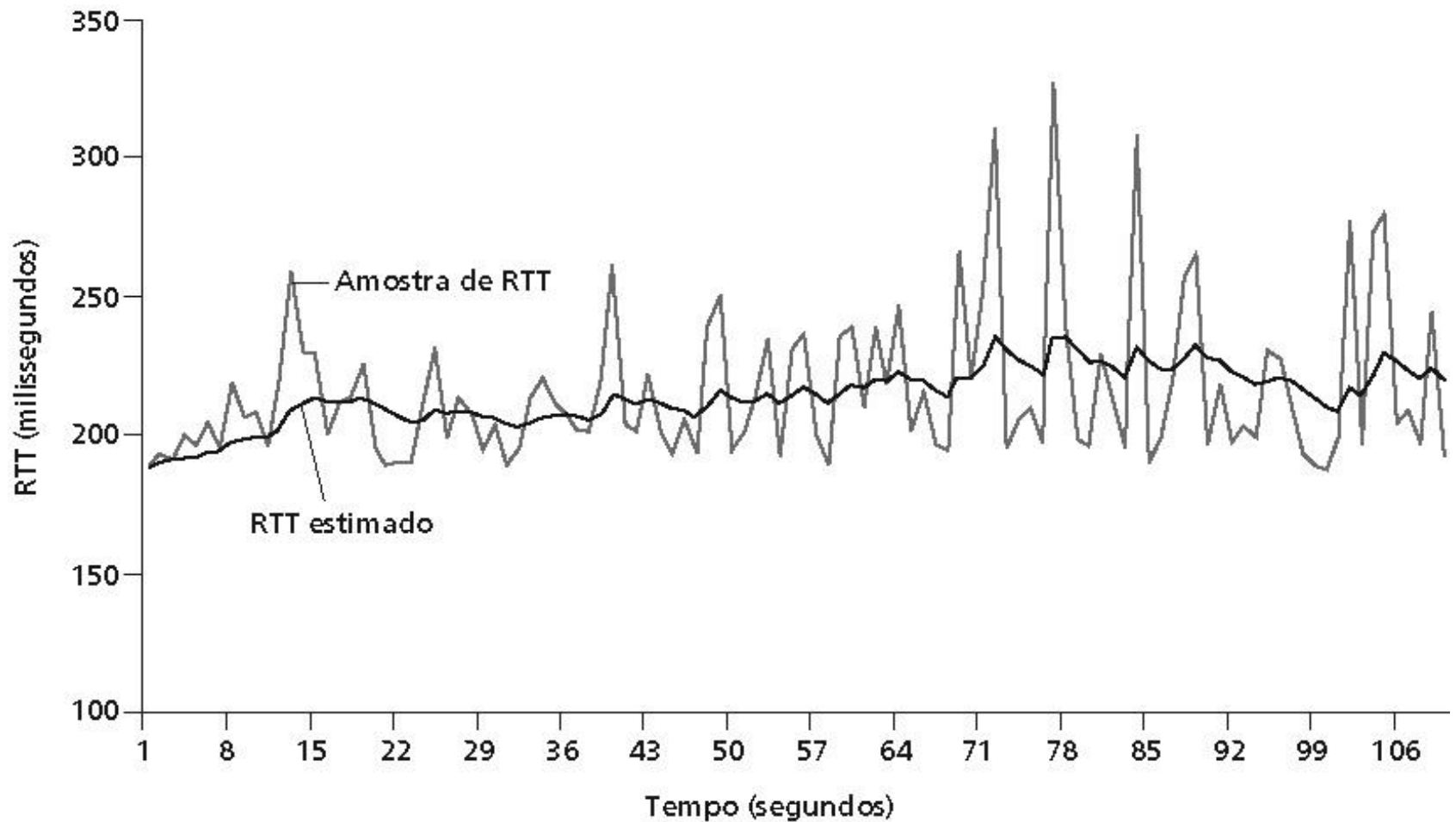
- **SampleRTT**: tempo medido entre a transmissão do segmento e o recebimento do ACK correspondente
 - ignora retransmissões
- **SampleRTT** varia de forma rápida, é desejável um “amortecedor” para a estimativa do RTT
 - usa várias medições recentes, não apenas o último **SampleRTT** obtido

TCP: Tempo de Resposta (RTT) e Temporização

$$\text{EstimatedRTT} = (1-\alpha) * \text{EstimatedRTT} + \alpha * \text{SampleRTT}$$

- ❑ média móvel exponencialmente ponderada
- ❑ influência de cada amostra diminui exponencialmente com o tempo
- ❑ valor típico de $\alpha = 0,125$

Exemplo de estimativa do RTT:



TCP: Tempo de Resposta (RTT) e Temporização

Escolhendo o intervalo de temporização

- **EstimatedRTT** mais uma “margem de segurança”
 - grandes variações no **EstimatedRTT**
 - maior margem de segurança
- primeiro estimar o quanto a **SampleRTT** se desvia do **EstimatedRTT**:

$$\text{DevRTT} = (1-\beta) * \text{DevRTT} + \beta * |\text{SampleRTT} - \text{EstimatedRTT}|$$

- (valor típico de $\beta = 0,25$)
Então, ajusta o temporizador para:

$$\text{TimeoutInterval} = \text{EstimatedRTT} + 4 * \text{DevRTT}$$

Conteúdo do Capítulo 3

- 3.1 Serviços da camada de transporte
- 3.2 Multiplexação e demultiplexação
- 3.3 Transporte não orientado para conexão: UDP
- 3.4 Princípios da transferência confiável de dados
- 3.5 Transporte orientado para conexão: TCP
 - estrutura do segmento
 - **transferência confiável**
 - controle de fluxo
 - gerenciamento de conexões
- 3.6 Princípios de controle de congestionamento
- 3.7 Controle de congestionamento do TCP

Transferência de dados confiável do TCP

- O TCP cria um serviço rdt sobre o serviço não confiável do IP
- Segmentos transmitidos em “paralelo” (*pipelined*)
- Acks cumulativos
- O TCP usa um único temporizador para retransmissões
- As retransmissões são disparadas por:
 - estouros de temporização
 - acks duplicados
- Considere inicialmente um transmissor TCP simplificado:
 - ignora acks duplicados
 - ignora controles de fluxo e de congestionamento

Eventos do transmissor TCP

Dados recebidos da aplicação:

- Cria segmento com no. de seqüência (nseq)
- nseq é o número de seqüência do primeiro byte de dados do segmento
- Liga o temporizador se já não estiver ligado (temporização do segmento mais antigo ainda não reconhecido)
- Valor do temporizador: calculado anteriormente

Estouro do temporizador:

- Retransmite o segmento que causou o estouro do temporizador
- Reinicia o temporizador

Recepção de Ack:

- Se reconhecer segmentos ainda não reconhecidos
 - atualizar informação sobre o que foi reconhecido
 - religa o temporizador se ainda houver segmentos pendentes (não reconhecidos)

TransmissorTCP (simplificado)

Comentário:

•SendBase-1: último byte reconhecido cumulativamente

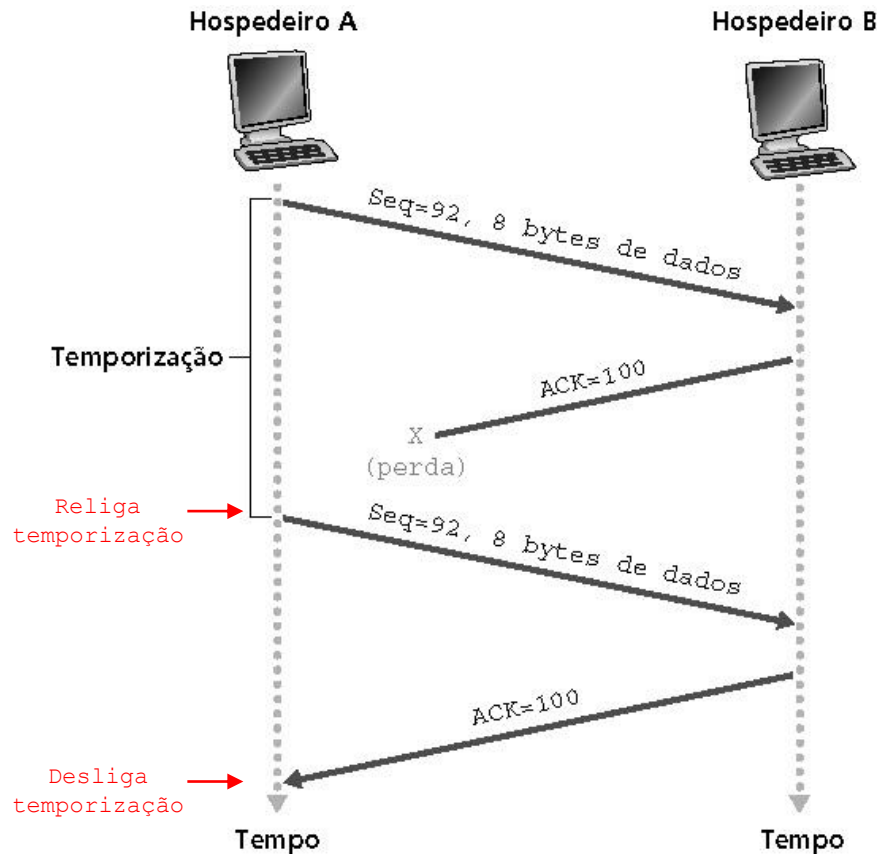
Exemplo:

•SendBase-1 = 71; y= 73, portanto o receptor quer receber 73+;

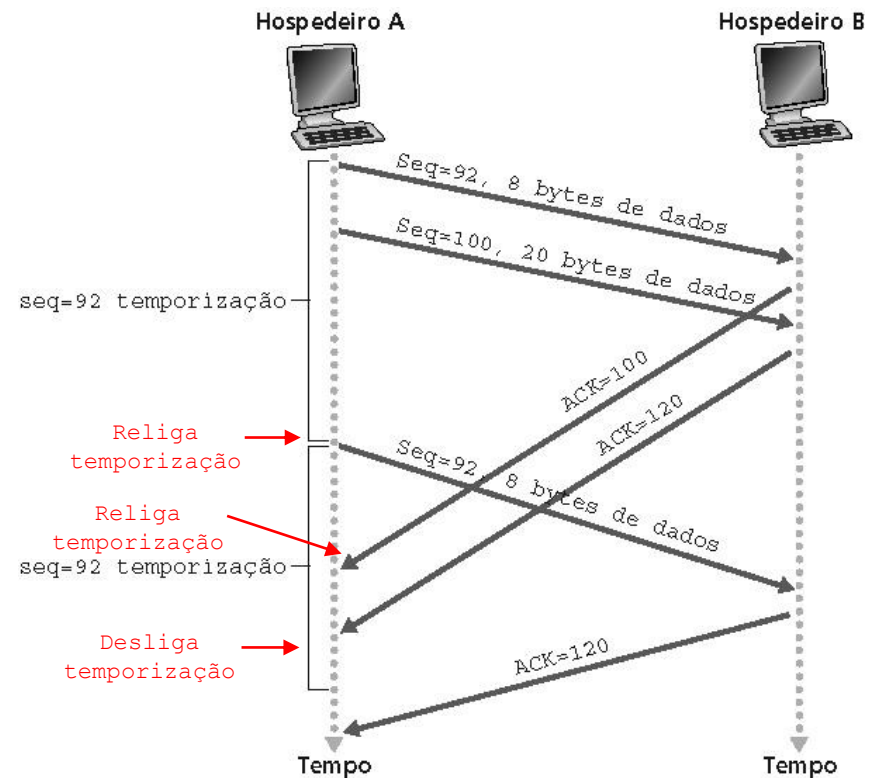
•y > SendBase, portanto novos dados foram reconhecidos.

- NextSeqNum = número de seqüência inicial
- SendBase = número de seqüência inicial
- repita (sempre) {
- switch(event)
- event: dados recebidos da aplicação acima
- cria segmento TCP com número de seqüência NextSeqNum
- se (temporizador estiver desligado)
- liga o temporizador
- passa segmento para IP
- NextSeqNum = NextSeqNum + comprimento(dados)
- event: estouro do temporizador
- retransmite segmento ainda não reconhecido com o menor número de seqüência
- reinicia o temporizador
- event: ACK recebido, com valor de campo ACK de y
- se (y > SendBase) { /* ACK cumulativo de todos dados até y */
- SendBase = y
- se (houver segmentos ainda não reconhecidos)
- liga o temporizador
- } senão desliga o temporizador
- } /* fim do repita sempre */

TCP: cenários de retransmissão

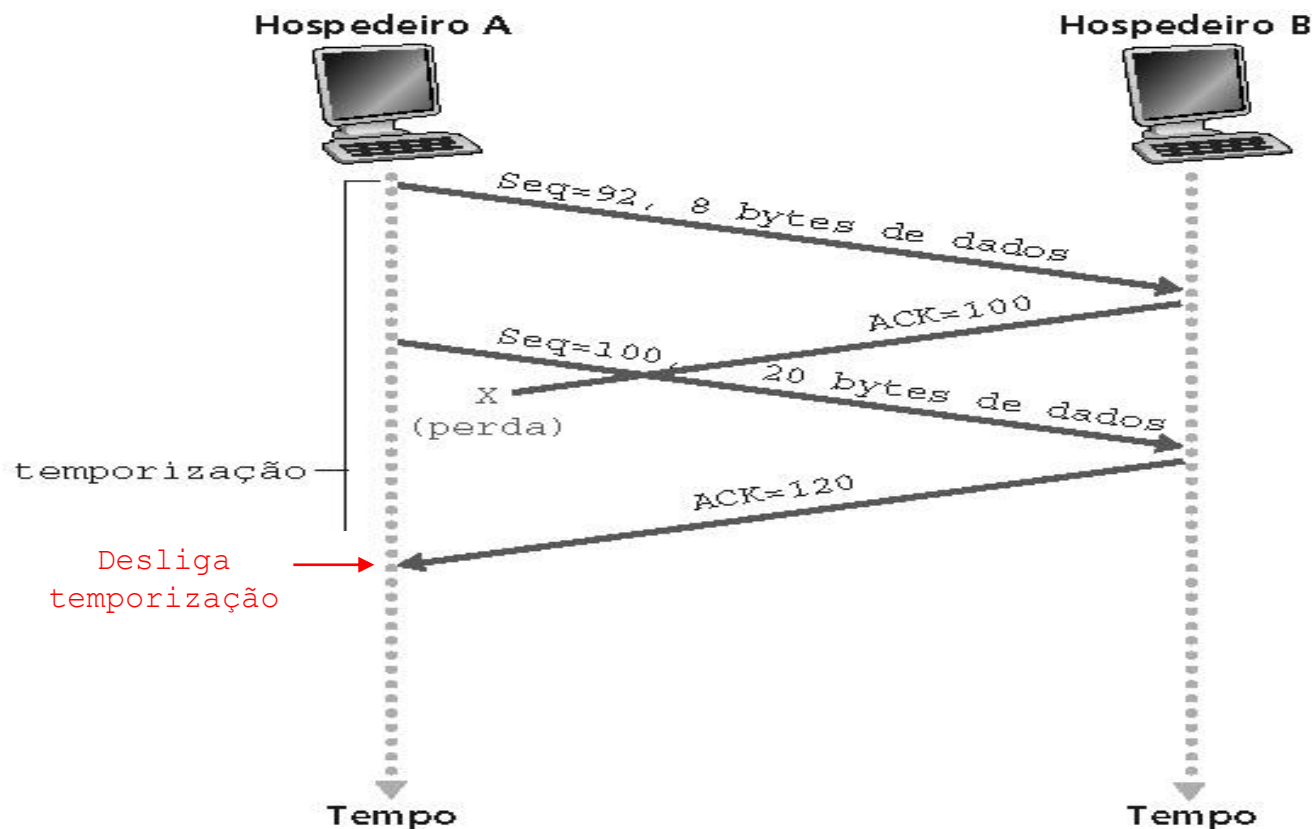


Cenário com perda do ACK



Temporização prematura, ACKs cumulativos

TCP: cenários de retransmissão (mais)



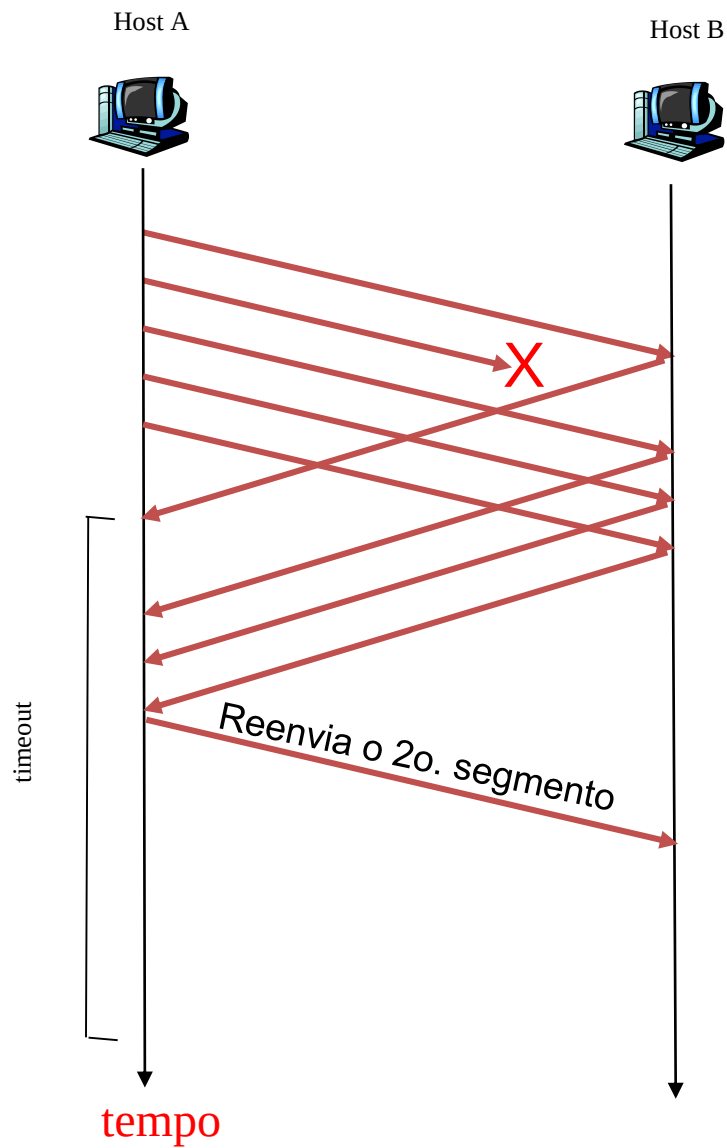
Cenário de ACK cumulativo

TCP geração de ACKs [RFCs 1122, 2581]

Evento no Receptor	Ação do Receptor TCP
chegada de segmento em ordem sem lacunas, anteriores já reconhecidos	ACK retardado. Espera até 500ms pelo próx. segmento. Se não chegar segmento, envia ACK
chegada de segmento em ordem sem lacunas, um ACK retardado pendente	envia imediatamente um único ACK cumulativo
chegada de segmento fora de ordem, com no. de seq. maior que esperado -> lacuna	envia ACK duplicado , indicando no. de seq.do próximo byte esperado
chegada de segmento que preenche a lacuna parcial ou completamente	ACK imediato se segmento começa no início da lacuna

Retransmissão rápida

- O intervalo do temporizador é freqüentemente bastante longo:
 - longo atraso antes de retransmitir um pacote perdido
- Detecta segmentos perdidos através de ACKs duplicados.
 - O transmissor normalmente envia diversos segmentos
 - Se um segmento se perder, provavelmente haverá muitos ACKs duplicados.
- Se o transmissor receber 3 ACKs duplicados para os mesmos dados, ele supõe que o segmento após os dados reconhecidos se perdeu:
 - Retransmissão rápida:
retransmite o segmento antes que estoure o temporizador



Retransmissão de um segmento após três ACKs duplicados

Algoritmo de retransmissão rápida:

event: recebido ACK, com valor do campo ACK de y

```
if (y > SendBase) {
```

```
    SendBase = y
```

```
    if (houver segmentos ainda não reconhecidos)
```

```
        liga temporizador
```

```
    else desliga temporizador
```

```
}
```

```
else {
```

```
    incrementa contador de ACKs duplicados recebidos para y
```

```
    if (contador de ACKs duplicados recebidos para y = 3) {
```

```
        retransmita segmento com número de seqüência y
```

```
    }
```

um ACK duplicado para um
segmento já reconhecido

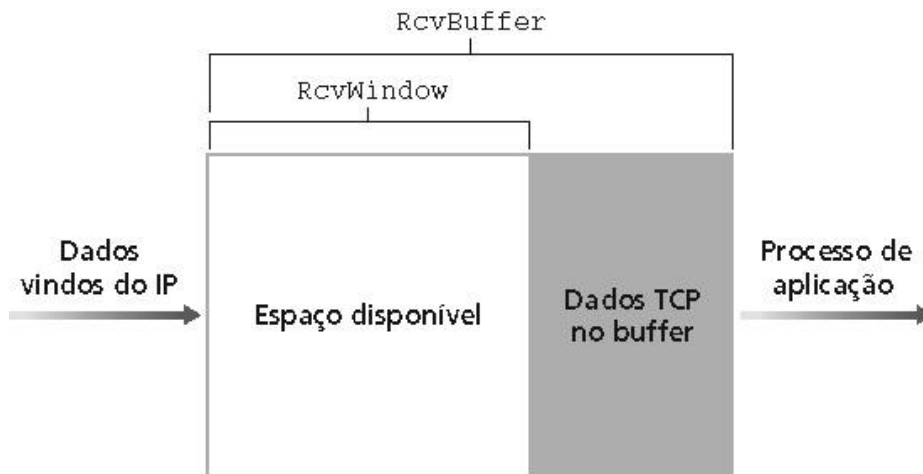
Retransmissão rápida

Conteúdo do Capítulo 3

- 3.1 Serviços da camada de transporte
- 3.2 Multiplexação e demultiplexação
- 3.3 Transporte não orientado para conexão: UDP
- 3.4 Princípios da transferência confiável de dados
- 3.5 Transporte orientado para conexão: TCP
 - estrutura do segmento
 - transferência confiável
 - controle de fluxo
 - gerenciamento de conexões
- 3.6 Princípios de controle de congestionamento
- 3.7 Controle de congestionamento do TCP

Controle de Fluxo do TCP

- Lado receptor da conexão TCP possui um buffer de recepção:



- serviço de compatibilização de velocidades: adaptando a taxa de transmissão à taxa de leitura da aplicação receptora

Controle de fluxo

o transmissor não inundará o buffer do receptor transmitindo muito e rapidamente

- ❑ Processos da aplicações podem demorar a ler do buffer

Controle de Fluxo do TCP: como funciona



- O receptor anuncia o espaço livre incluindo o valor da **RcvWindow** nos segmentos
- O transmissor limita os dados não reconhecidos ao tamanho da **RcvWindow**
 - Garante que o buffer do receptor não transbordará

(Suponha que o receptor TCP descarte segmentos fora de ordem)

- espaço livre no buffer

= **RcvWindow**

= **RcvBuffer - [LastByteRcvd - LastByteRead]**

Conteúdo do Capítulo 3

- 3.1 Serviços da camada de transporte
- 3.2 Multiplexação e demultiplexação
- 3.3 Transporte não orientado para conexão: UDP
- 3.4 Princípios da transferência confiável de dados
- 3.5 Transporte orientado para conexão: TCP
 - estrutura do segmento
 - transferência confiável
 - controle de fluxo
 - gerenciamento de conexões
- 3.6 Princípios de controle de congestionamento
- 3.7 Controle de congestionamento do TCP

TCP: Gerenciamento de Conexões

Lembrete: Transmissor e receptor TCP estabelecem “conexão” antes de trocar segmentos de dados

- inicializam variáveis TCP:
 - números de seqüência
 - buffers, info s/ controle de fluxo (ex.: **RcvWindow**)
- *cliente*: iniciador de conexão

```
Socket clientSocket = new
Socket("hostname", "port
number");
```
- *servidor*: contactado pelo cliente

```
Socket connectionSocket =
welcomeSocket.accept();
```

Apresentação de 3 etapas:

Etapa 1: sistema cliente envia segmento de controle SYN do TCP ao servidor

- especifica no. inicial de seq
- não envia dados

Etapa 2: sistema servidor recebe SYN, responde com segmento de controle SYNACK

- aloca buffers
- especifica no. inicial de seq. servidor-> receptor

Etapa 3: receptor recebe SYNACK, responde com segmento ACK que pode conter dados.

TCP: Gerenciamento de Conexões (cont.)

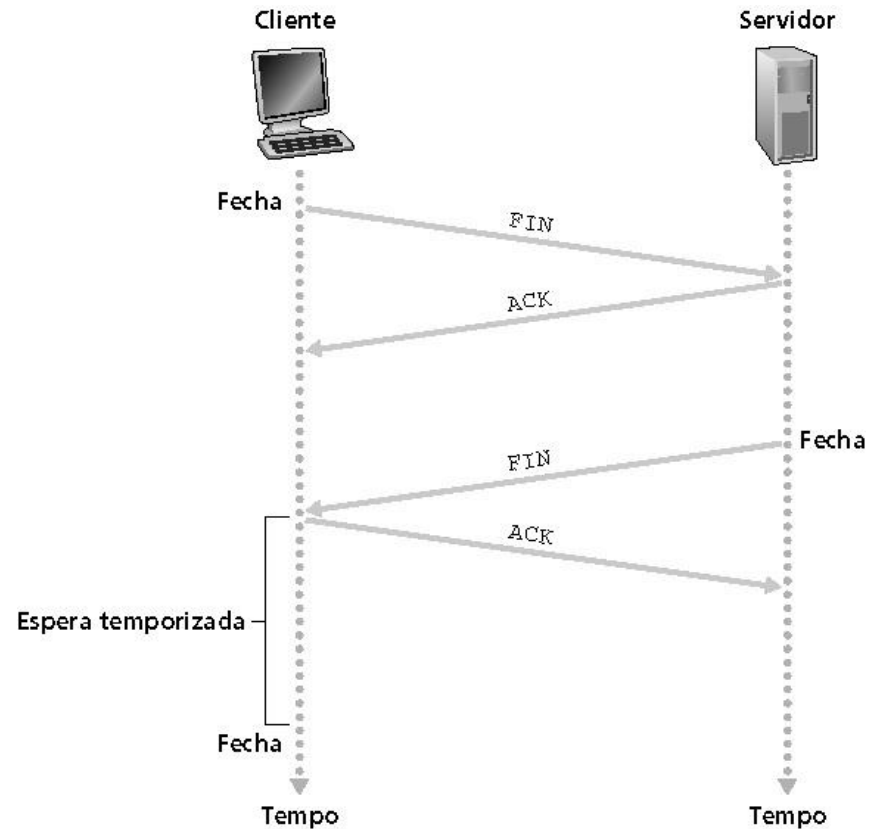
Encerrando uma conexão:

cliente fecha o socket:

```
clientSocket.close();
```

Passo 1: sistema **cliente** envia segmento de controle FIN ao servidor

Passo 2: **servidor** recebe FIN, responde com ACK. Encerra a conexão, enviando FIN.



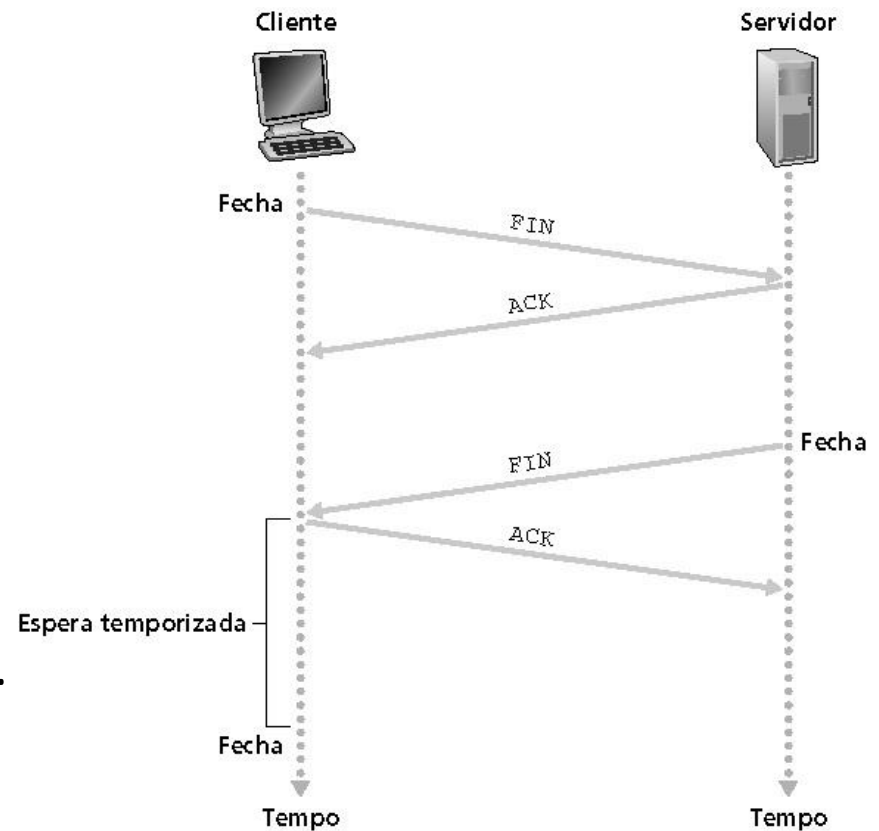
TCP: Gerenciamento de Conexões (cont.)

Passo 3: cliente recebe FIN,
responde com ACK.

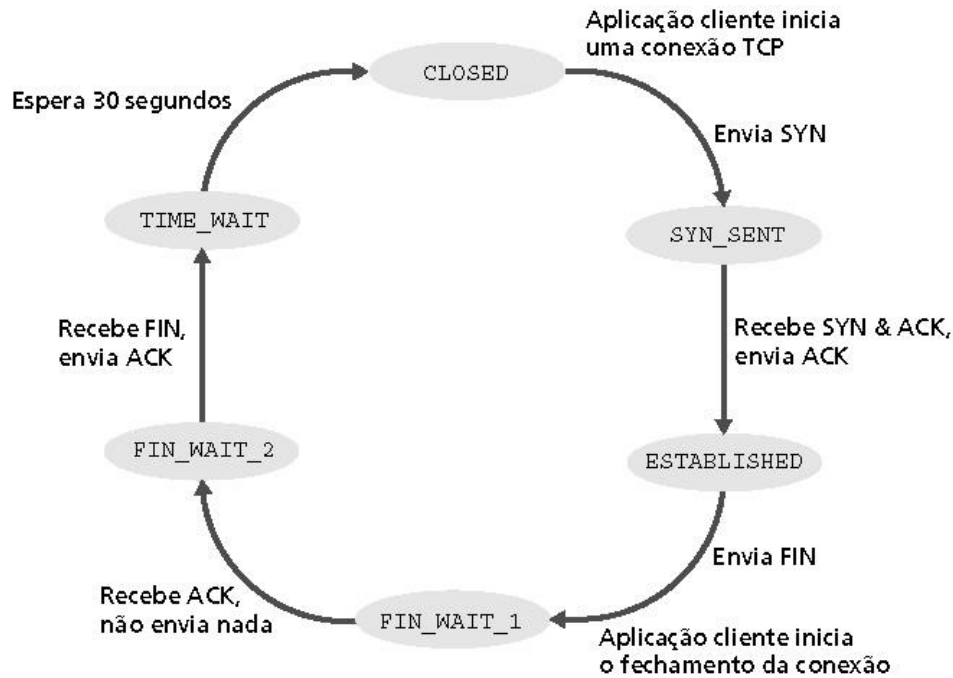
- Entre em “espera temporizada” - responderá com ACK a FINs recebidos

Passo 4: servidor, recebe ACK.
Conexão encerrada.

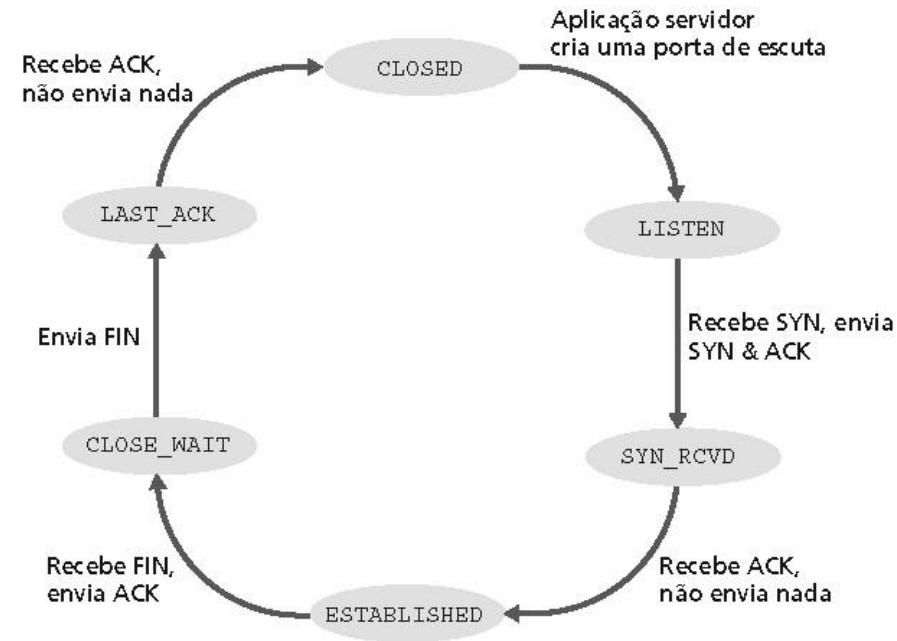
Nota: com pequena modificação,
consegue tratar FINs simultâneos.



TCP: Gerenciamento de Conexões (cont.)



Estados do cliente



Estados do servidor